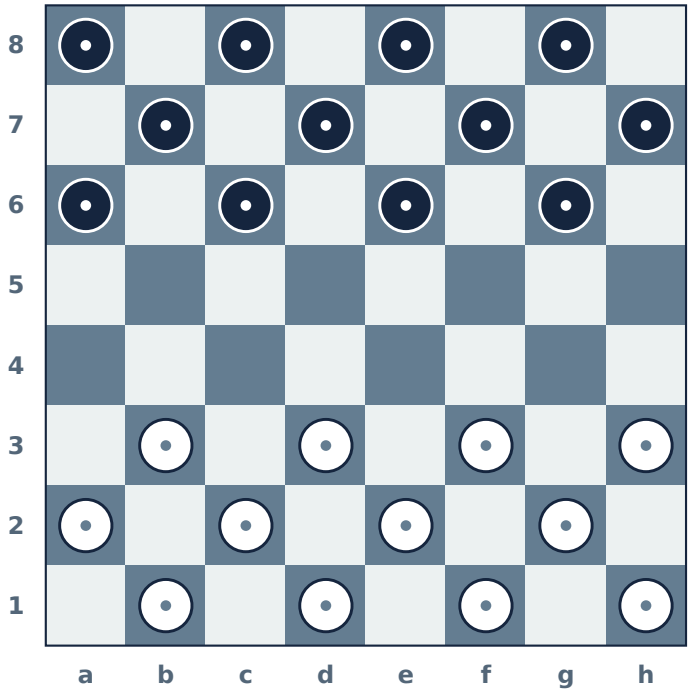
Build Checkers in C

A console game, built in small stages | Student exercise

THE CHALLENGE

Make a two-player checkers game in one `checkers.c` file. The players enter moves in the terminal. White plays first, then Black, then White again. The game must explain invalid moves and enforce captures, kings, and a winner.

See the board



Light square

Pale square: no piece may stand here.

Dark square

Blue-gray square: moves and pieces use these squares.

Pieces

White disks begin on rows 1–3. Dark disks begin on rows 6–8.

Coordinates

Letters run a–h from left to right. Numbers run 1–8 from bottom to top. b3 names the dark square in column b, row 3.

How a move is entered

Type the starting square, a space, and the destination: `b3 a4`. That moves the White piece from b3 to a4. A capture also uses the same format; for example, `b3 d5` can jump over a Black piece on c4 when d5 is empty.

Build and run on Linux

Save the program as `checkers.c`. From the folder containing it, run:

```
gcc -std=c11 -Wall -Wextra -Wpedantic checkers.c -o checkers
./checkers
```

The first command compiles and names the binary `checkers` (the `-o checkers` part); the second runs it. The test script looks for `./checkers` by default. For another name, run `CHECKERS_BIN=./my_game python3 -m pytest -v test_checkers.py` (or prefix a stage command the same way). Recompile after each change. Keep `test_checkers.py` and `requirements.txt` beside `checkers.c`. Install pytest once with `python3 -m pip install -r requirements.txt`. If GCC is missing on Ubuntu/Debian, install `build-essential`.

Begin with turns and movement

Reach a simple playable game before adding captures.

Each stage ends with a test command. The `-k stage1` part selects only that stage's checks; change the number as you progress. Rebuild the binary before running tests after a code change.

Stage 1 | Set up the board

- Show the 8 × 8 board with visible light and dark squares, row numbers, column letters, and the starting positions shown on page 1.
- Only dark squares can hold pieces. Start with 12 White pieces and 12 Black pieces; the middle two rows are empty.

READY WHEN

Someone looking at the terminal can find b3 and a4 and tell which pieces belong to White and Black.

Check the board visually, then run: `python3 -m pytest -v test_checkers.py -k stage1`

Stage 2 | Let two people take turns

- Ask White for a move first. Accept one move per line as two coordinates, such as `b3 a4`. After a valid move, show the new board and ask Black. Continue alternating.
- If the text is unclear, a square is outside the board, the source is empty or belongs to the other player, or the destination is occupied, explain the problem and ask the same player again. The board must stay as it was.

TRY IT

White enters `b3 a4`. Black can then enter `a6 b5`. If Black instead types an invalid move, it remains Black's turn.

Run this stage: `python3 -m pytest -v test_checkers.py -k stage2`

Stage 3 | Allow ordinary moves

- An ordinary piece goes one dark square diagonally forward into an empty square. White moves toward row 8; Black moves toward row 1. It cannot move sideways, straight ahead, backward, or through an occupied destination.
- For now, players do not capture. The game should reject a two-square move until the next stage.

TRY IT

At the beginning, White's `b3 a4` is legal. `b3 b4` is straight ahead, `b3 c2` goes backward, and `b3 d5` is too far for an ordinary move. Each is rejected.

Run this stage: `python3 -m pytest -v test_checkers.py -k stage3`

Make captures matter

The game now has to look beyond the player's chosen move.

Stage 4 | Capture an opponent

- A capture jumps diagonally over one adjacent opponent piece and lands on the empty dark square immediately beyond it. Remove the jumped piece. Ordinary pieces capture forward only.
- Reject a jump if the middle piece is friendly or missing, the landing square is occupied or off the board, or the jump goes backward for an ordinary piece. A rejected jump changes nothing.

> **SMALL TEST POSITION**
>
> Imagine an otherwise empty board with White at `b3` and Black at `c4`. White can enter `b3 d5`; the Black piece on `c4` disappears.

Run this stage: `python3 -m pytest -v test_checkers.py -k stage4`

Stage 5 | Require available captures

- Before accepting a move, consider every piece of the player whose turn it is. If at least one capture is available, a normal one-square move is forbidden. The player chooses one of the available captures.
- After a capture, if that same piece can capture again, keep the turn with the same player and require the next jump with that piece. The turn ends only when its chain is finished.

> **CHAIN TEST POSITION**
>
> On an otherwise empty board, put White at `b3` and Black at `c4` and `e6`. White must play `b3 d5`, then `d5 f7`. Black does not play between these jumps.

Run this stage: `python3 -m pytest -v test_checkers.py -k stage5`

Stage 6 | Crown kings

- When a White ordinary piece reaches row 8, or a Black one reaches row 1, crown it as a king. Make kings visibly different on the printed board.
- A king may move one square diagonally or capture by jumping in either direction. It still obeys mandatory captures and chained jumps.
- If a jump reaches the far edge and crowns an ordinary piece, that player's turn ends immediately, even if the new king would have another jump.

Run this stage: `python3 -m pytest -v test_checkers.py -k stage6`

Finish and check the game

The final stage defines when a player has lost.

Stage 7 | Announce a winner

- A player loses if they have no pieces left or cannot make any legal move when their turn begins. Announce the winner and stop asking for moves.
- If terminal input ends, exit cleanly. Draws, time limits, and resignation are outside this exercise.

> **THE TURN RULE**
>
> A turn passes to the other player only after a valid, fully finished move or capture chain. Mistakes never cost a turn. White always starts.

Run this stage: `python3 -m pytest -v test_checkers.py -k stage7`

Then run every check: `python3 -m pytest -v test_checkers.py`. A failure shows the expected and observed turns and recent game output. Fix the program, rebuild, and rerun.

Playtest checklist

- ☐ The initial board distinguishes pale light squares from dark playable squares and has 12 pieces on each side.
- ☐ White goes first; completed turns alternate White, Black, White, Black.
- ☐ Bad input, wrong-color pieces, occupied destinations, and moves off the board are rejected without changing the turn.
- ☐ Ordinary pieces move and capture forward; kings work in both directions.
- ☐ A jump removes the enemy piece, but a friendly piece cannot be jumped.
- ☐ A normal move is rejected when any capture is available, even from another piece.
- ☐ After a jump, another available jump with that piece is required before changing turns.
- ☐ Promotion is visible and ends that turn; no-piece and no-move losses end the game.

Useful test positions

In addition to a full game from the starting board, try a nearly empty board with only the pieces needed for a case. Check the two-jump position on page 3, a king moving backward, a blocked player, and a capture that reaches the crowning row. Restore the usual starting board for the finished game.

For the companion automated checker

The checker does not read the board display; arrange that however you like. For each input request, print a recognizable player prompt such as `White>` or `Black>`. Repeat the same player's prompt after invalid input and between required jumps. After a finished turn, prompt the other player. Exit on Ctrl+D (EOF), and announce a win with text such as `White wins`. The script defaults to those prompt words and lets you change the matching expressions if needed.

Optional extensions

When the core game works, consider move history, save/load, or a draw rule. These are extras, not requirements.